

Железо для ИИ: от задачи к конфигурации

Покупка начинается с измеримой работы. Одна и та же модель предъявляет разные требования при одиночном разговоре, многопользовательском сервисе и дообучении.

Что именно должна сделать машина

В этом выпуске построим локальную систему для работы с документами: загрузка PDF и таблиц, OCR, извлечение фрагментов, поиск, переранжирование и ответ с цитатами. Иногда владелец хочет дообучить генератор или модель эмбедингов. Для учебного расчёта возьмём условный плотный генератор с 8 млрд параметров, 32 слоями, восемью KV-головами размером 128; удерживаемый контекст — 4096 токенов. Позже сравним один и двадцать активных запросов и корпус из 150 тысяч фрагментов. Эти числа задают опыт, а не описывают конкретный checkpoint.

Сначала составим паспорт нагрузки. Укажем точные ревизии генератора, OCR, embedding и reranker; количество и тип документов; распределение длин запроса и ответа; число одновременно активных генераций. Определим допустимое время до первого токена, скорость дальнейшей выдачи и качество ответов с правильными страницами источников. Отдельно запишем, что данные должны оставаться на собственной машине. Эти ограничения превращаются в условия, по которым можно испытать конфигурацию.

Пять путей через одну систему

При индексации CPU разбирает форматы и очередь, OCR может использовать GPU, SSD подаёт страницы, а RAM удерживает промежуточные тексты. Embedding-модель превращает фрагменты в векторы; база хранит их вместе с метаданными. Запрос пользователя сначала ищет кандидатов, затем генератор получает выбранные фрагменты. *Prefill* обрабатывает входные токены, *decode* создаёт выходные по одному. Обучение добавляет градиенты, активации, оптимизатор и запись checkpoint.

GPU важен не на каждой стадии. Медленный парсер или база, занятая восстановлением индекса, не ускорятся от более быстрой видеокарты. И наоборот, достаточный CPU не компенсирует нехватку памяти под веса и активные последовательности. Поэтому паспорт делится на пять строк: подготовка корпуса, поиск, одиночный инференс, сервис под нагрузкой и адаптация модели. Для каждой строки нужны своя единица результата и своё ограничение по времени.

Вместимость и производительность

«Загрузилось» означает лишь, что размещение удалось в заданном режиме. Часть весов может находиться в RAM или на диске, а кэш при реальных запросах ещё не выделен. «Годится» означает, что точная задача работает при целевых задержке, качестве и устойчивости. Это различие проведём через все расчёты выпуска.

Три разных вопроса

1. Вместимость

Поместятся ли веса, кэш и рабочие буферы при 4096 токенах?

2. Скорость

Уложится ли ответ в заданную задержку при двадцати активных запросах?

3. Качество

Останутся ли правильными ответы и страницы цитат?

Что прочитать сначала

CUDA Programming Guide¹, Programming Model и Understanding Memory: проследить один запрос через CPU, RAM и GPU.

Граница главы

Расчёты учебные; цена и поддержка точного checkpoint проверяются на дату испытания.

Подтемы главы

1. Архитектура узла
2. Память модели
3. Обучение
4. Инференс
5. Данные и индексы
6. Локальные модели
7. Выбор компонентов

От паспортных чисел к узкому месту

CUDA Programming Guide¹ описывает иерархию памяти и исполнение ядер. Внутри ускорителя вычисления могут опережать подачу данных. Best Practices Guide² учит измерять фактические передачи между CPU и GPU. Roofline³ предлагает модель: если для одного байта перемещаемых данных выполняется мало операций, верхняя граница задаётся bandwidth; при большей интенсивности — вычислительным пиком. Формально, для ядра с интенсивностью I производительность ограничена $\min(P_{\max}, I B_{\max})$. Это верхняя граница при заданных предположениях, а не обещание скорости целого LLM-сервера.

FlashAttention⁴ показывает, что способ перемещения данных может быть важнее числа арифметических действий: промежуточные матрицы не нужно полностью записывать в дорогую память. Но и это не означает одинакового выигрыша при коротком входе, другой архитектуре или неподдерживаемом ядре. По той же причине таблица FLOPS не даёт прямой ранжировки машин для интерактивной генерации.

Как выбирать между классами машин

Для одиночного исследования начните с имеющейся машины и небольшой открытой модели. Если веса с рабочим кэшем помещаются в доступную память и результат отвечает требованиям, рост класса GPU необязателен. При большом числе параллельных запросов проверьте суммарный KV и p95 задержки. При редком полном обучении сравните постоянную покупку кластера с арендой на время эксперимента. Локальный RAG может больше выиграть от памяти и надёжного SSD под корпус, чем от добавления второго GPU.

Решение оформляется проверяемым протоколом: версия модели и квантизации, версия программного стека, размеры входов, распределение поступления запросов, измеренные задержки и качество. Цена покупки — только одна часть стоимости; добавляются электричество, хранение, эксплуатация и возможный простой. Здесь не даём готовой корзины устройств, потому что совместимость и рыночная цена меняются. Вместо этого в конце выпуска читатель составит три допустимых конфигурации и протестирует точные модели.

Маршрут выпуска

Сначала разберём движение данных и памяти. Затем оценим веса и KV, расходы обучения, работу сервера, индекс и локальный запуск. Последняя часть сведёт расчёты к выбору CPU, GPU, RAM, SSD, платы, питания и сети. Каждый вывод будет иметь три уровня: нижняя оценка по формуле, проверка поддержки и фактический замер на задаче.

Roofline по шагам

1. Измерить

Сколько операций и байтов проходит через выбранный уровень памяти?

2. Разделить

Интенсивность
 $I = \text{операции/байты}$.

3. Сравнить

$I B_{\max}$ и $P_{\max}$: меньшая величина задаёт потолок ядра, не сервиса.

Где разобрано

Roofline³: operational intensity и границы. FlashAttention⁴: как tiling сокращает передачи в HBM.

Проверить на себе

Если ответ долго ждёт очередь и поиск, почему ускорение одного CUDA-ядра не обязано уменьшить общую задержку?

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. NVIDIA (документация, 2026), *CUDA Programming Guide*. Документация.

Руководство по программированию CUDA

Описаны устройство вычислительных блоков, пространства памяти, запуск ядер и программная модель CUDA. Термины помогают отделить локальность данных от паспортного числа операций в секунду.

Что читать. Programming Model; Understanding Memory; Features and Technical Specifications. Нарисовать путь данных от RAM до регистров.

2. NVIDIA (документация, 2026), *CUDA C++ Best Practices Guide*. Документация.

Практическое руководство по производительности CUDA

Документация разбирает передачу данных, локальность, coalescing и профилирование. Полезна для понимания, почему фактическая пропускная способность ниже рекламируемой и когда загрузка GPU не означает эффективную работу.

Что читать. Data Transfer Between Host and Device; Memory Optimizations; Performance Metrics. Сравнить requested и actual bandwidth.

3. Williams, S.; Waterman, A.; Patterson, D. (2009), *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures*. Исследовательская статья.

Roofline: модель верхних границ вычислений и обмена с памятью

Связывает достижимую производительность ядра с арифметической интенсивностью, пиком вычислений и пропускной способностью памяти. Даёт язык для гипотезы об узком месте, но не прогнозирует задержку целого сервиса.

Что читать. Разделы о модели и operational intensity; построить две границы для одной операции, указать допущения.

4. Dao, T. и др. (2022), *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS.

FlashAttention: точное внимание с учётом обмена между уровнями памяти

Разбиение вычисления attention на блоки уменьшает движение данных между HBM и быстрой памятью GPU. Это пример ускорения без изменения математической цели attention; эффект зависит от формы задачи и реализации.

Что читать. Постановка IO-aware attention и раздел Algorithm; объяснить, какие промежуточные данные перестают записываться в HBM.

Порядок чтения. CUDA даёт аппаратный словарь, Best Practices — измерение передач. Затем построить простой Roofline для одного ядра. Завершить FlashAttention как пример изменения алгоритма обращения к памяти. После чтения описать узкое место собственного процесса в проверяемой форме.

1. Архитектура узла и иерархия памяти

CPU, RAM, ускоритель, SSD и сеть образуют путь данных. Баланс этих звеньев зависит от того, какие операции выполняются и где находятся данные.

Машина — это путь, а не список деталей

CPU управляет процессами, токенизацией, парсингом, очередями и частью численных операций. RAM удерживает документы, индексные структуры, модель при выгрузке и файловый кэш. GPU быстро выполняет большие массивы операций, используя собственную VRAM или общую память в некоторых системах. SSD предоставляет checkpoint и корпус; сеть доставляет документы, запросы и данные при распределённом обучении. У каждого перехода есть задержка и пропускная способность.

Рассмотрим запрос к документной системе. Даже если матричные умножения занимают миллисекунды, поиск может ждать страницы с диска, а передача результата reranker в контекст генератора — CPU. Большой объём RAM полезен, когда индекс и рабочий корпус помещаются в ней; большая VRAM нужна, когда мы удерживаем веса и KV на GPU. Подмена одной памяти другой возможна только если runtime поддерживает перенос и задержка остаётся приемлемой.

CPU и системная память

Для CPU сравнивайте число ядер, производительность одного потока, поддерживаемые инструкции, каналы RAM и пропускную способность памяти. OCR и загрузка многих документов могут хорошо распараллеливаться, тогда как синхронный участок запроса ограничивает общую задержку. Для RAM важны ёмкость, bandwidth, возможность расширения и, в сервере, требования к ECC. NUMA означает, что доступ к памяти другого сокета может стоить дороже локального; размещение процесса и данных имеет значение.

При выборе SSD запишите размер исходного корпуса, производных фрагментов, индекса, моделей, журналов и резервных копий. Последовательная скорость из рекламы не описывает чтение множества мелких файлов или восстановление после сбоя. Отдельный ресурс нужен для временных преобразований и свободного места при пересоздании индекса. Измерения делайте после прогрева и при типичном количестве одновременных задач.

GPU и размещение тензоров

Руководство CUDA¹ различает регистры, быструю память на чипе и глобальную память устройства. VRAM ёмкость отвечает на вопрос о размещении, её bandwidth — на вопрос о подаче данных, а вычислительные блоки и поддерживаемые численные форматы — о скорости подходящих ядер. Ширина шины в битах сама по себе не заменяет измеренную пропускную способность памяти.

Трасса запроса

1. Чтение

Фрагменты и метаданные из SSD либо файлового кэша RAM.

2. Отбор

CPU исполняет фильтр, индекс и токенизатор; reranker может занимать GPU.

3. Генерация

Веса и KV читаются из памяти ускорителя; ответ уходит через CPU и сеть.

Где искать механизм

CUDA Programming Guide¹, Understanding Memory: отделить память GPU от RAM и быстрой памяти на чипе.

Контрольный вопрос

Какой шаг продолжит ждать, если удвоить только число вычислительных блоков GPU?

Передачи и реальные пределы

CUDA Best Practices Guide² предлагает отдельно смотреть host-device transfer и локальность доступов. Если при decode часть слоёв лежит в RAM, эти веса снова проходят через узкий канал или исполняются на CPU. Иногда такой режим делает модель доступной, но худшее время ответа меняет пригодность машины. В кластере аналогичную роль играет межсоединение GPU и сеть: наличие двух ускорителей не означает, что произвольный checkpoint автоматически исполняется вдвое быстрее.

Roofline³ помогает классифицировать операцию. У prefill крупные матричные операции могут эффективно использовать вычислительные блоки. При одиночном decode полезная работа на загружаемый блок весов меньше, поэтому memory bandwidth часто важнее. Это качественное правило, а не универсальная диагностика: размер batch, архитектура, квантование и реализация ядра меняют интенсивность.

Для конкретной передачи объёма D по каналу с эффективной скоростью B_{eff} оптимистичная нижняя граница времени равна D/B_{eff} . Если через канал нужно провести 4 GB, а фактически доступно 400 GB/s, одно чтение заняло бы не меньше 0,01 секунды. Реальный токен содержит и другие передачи, операции и накладные расходы. Это не прогноз скорости модели: цель примера — понять, почему ещё один вычислительный блок не ускорит насыщенный канал.

Питание, охлаждение и физическая совместимость

Проверка железа не заканчивается названием GPU. Для настольной машины запишите физическую длину и толщину карты, расположение разъёмов питания, мощность БП с запасом под длительную нагрузку и возможность отвести тепло от GPU, CPU, RAM и SSD. Для сервера добавьте устойчивость питания, дистанционное управление, сетевые порты и путь замены диска. Машина, которая выдерживает пятиминутный тест, может снижать частоты через час или падать при долгом обучении.

Проверяйте *совместимость стека*: ОС, драйвер, runtime, версия библиотек, поддержка формата весов и операций конкретной модели. Если аппаратное устройство существует в матрице поставщика, но требуемое ядро отсутствует в выбранном приложении, задача не решена. Зафиксируйте версии и короткий пробный запуск прежде, чем считать конфигурацию подходящей.

Небольшой опыт

Запустите одинаковый вход после прогрева и измерьте отдельно чтение документа, поиск, prefill и decode. Затем удвойте только длину входа и только длину выхода. Если меняется главным образом TTFT, исследуйте prefill и поиск; если скорость токенов падает при росте числа запросов, смотрите KV, batch и bandwidth. FlashAttention⁴ служит напоминанием: перенос данных можно уменьшить алгоритмом, не покупая новый ускоритель.

Нижняя граница времени

Условно $D = 4$ GB и $B_{\text{eff}} = 400$ GB/s. Тогда $D/B_{\text{eff}} = 0,01$ s. Это лишь одно чтение, без поиска, очереди и вычислений.

Что измерять отдельно

Подача

Занятость CPU, SSD, PCIe и GPU; Best Practices², Data Transfer Between Host and Device.

Долгая работа

Память, мощность, частоты и р95 после прогрева; наблюдать снижение частоты.

Различающий опыт

Оставить checkpoint тем же и поменять только размещение весов. Если задержка резко меняется, исследовать перенос и offload до покупки нового GPU.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. NVIDIA (документация, 2026), *CUDA Programming Guide*. Документация.

Руководство по программированию CUDA

Описаны устройство вычислительных блоков, пространства памяти, запуск ядер и программная модель CUDA. Термины помогают отделить локальность данных от паспортного числа операций в секунду.

Что читать. Programming Model; Understanding Memory; Features and Technical Specifications. Нарисовать путь данных от RAM до регистров.

2. NVIDIA (документация, 2026), *CUDA C++ Best Practices Guide*. Документация.

Практическое руководство по производительности CUDA

Документация разбирает передачу данных, локальность, coalescing и профилирование. Полезна для понимания, почему фактическая пропускная способность ниже рекламируемой и когда загрузка GPU не означает эффективную работу.

Что читать. Data Transfer Between Host and Device; Memory Optimizations; Performance Metrics. Сравнить requested и actual bandwidth.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Williams, S.; Waterman, A.; Patterson, D. (2009), *Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures*. Исследовательская статья.

Roofline: модель верхних границ вычислений и обмена с памятью

Связывает достижимую производительность ядра с арифметической интенсивностью, пиком вычислений и пропускной способностью памяти. Даёт язык для гипотезы об узком месте, но не прогнозирует задержку целого сервиса.

Что читать. Разделы о модели и operational intensity; построить две границы для одной операции, указать допущения.

4. Dao, T. и др. (2022), *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS.

FlashAttention: точное внимание с учётом обмена между уровнями памяти

Разбиение вычисления attention на блоки уменьшает движение данных между HBM и быстрой памятью GPU. Это пример ускорения без изменения математической цели attention; эффект зависит от формы задачи и реализации.

Что читать. Постановка IO-aware attention и раздел Algorithm; объяснить, какие промежуточные данные перестают записываться в HBM.

Порядок чтения. Сначала CUDA и Best Practices для пути данных. Затем Roofline, чтобы построить гипотезу об узком месте. FlashAttention использовать как контрольный пример алгоритмической альтернативы. После этого измерить конкретный этап своего процесса.

2. Сколько памяти требует модель

Оценка вместимости начинается с весов, но заканчивается кэшем, буферами и фактическим распределением запросов. Число параметров не равно расходу VRAM.

Нижняя граница для плотных весов

Если плотная модель имеет P параметров и каждый занимал бы ровно b бит, сырые веса потребовали бы $W_0 = Pb/8$ байт. Для $P = 8$ миллиардов и $b = 4$ получаем 4 миллиарда байт, или около 3,73 GiB. Это нижняя оценка, а не гарантированный размер файла: схемы квантования хранят масштабы и другие метаданные по блокам; отдельные слои могут оставаться в другом формате.

Нельзя смешивать GB и GiB: первый равен 10^9 байт, второй 2^{30} . Сравнение «4 GB весов» с «4 GiB свободной памяти» без учёта runtime и кэша ошибочно. Для MoE различают общее число параметров, которые надо хранить или подгружать, и число активных параметров при обработке токена. Слова «активно 8B» не означают, что достаточно памяти как для плотной модели 8B.

Кэш внимания

В обычном decoder-only Transformer без оконной схемы нижнюю оценку полного KV-кэша можно записать так:

$$K = 2LH_{KV}DSBs.$$

Здесь L — слои, H_{KV} — число голов ключей/значений, D — размер головы, S — удерживаемые токены одной последовательности, B — число активных последовательностей, s — байт на элемент. Множитель 2 относится к K и V. Подстановка учебной архитектуры приведена справа. Рост конкурентности B линейно увеличивает этот сырой массив, хотя реальный планировщик может разделять префиксы и размещать кэш блоками.

Документация о кэше¹ помогает проверить, что эта формула применима. У GQA в H_{KV} подставляются KV-головы, а не query-головы. Скользящее окно, MLA, сокращение кэша, prefix sharing и разные точности меняют картину. Нужна конфигурация именно выбранной архитектуры и версия runtime, а не одна универсальная таблица по числу параметров.

В расчёте B — одновременно удерживаемые последовательности, а не число зарегистрированных пользователей. При последовательной обработке тех же двадцати запросов пик KV может быть меньше, но общее время ожидания возрастёт. Если сервер допускает одновременную генерацию, исходный паспорт нагрузки должен задавать не только максимальный контекст, но и распределение фактически занятых токенов. Проверка по одному короткому запросу не показывает, сколько памяти потребуется в пике многопользовательского режима.

Память по шагам

1. Веса

8B × 4 бита: 4 GB ≈ 3,73 GiB без метаданных.

2. KV для одного

$2 \cdot 32 \cdot 8 \cdot 128 \cdot 4096 \cdot 2 = 2^{29}$ байт = 0,5 GiB.

3. Двадцать

Полный сырой KV — 10 GiB; веса + KV ≈ 13,73 GiB ещё без буферов.

Проверить формулу

Caching¹: тип кэша, GQA, окно внимания и offload выбранной модели. Число query-голов здесь не подставлять вместо H_{KV} .

Рабочая память и пик

Файл модели, занятая устройством память и память под один запрос — разные числа. При загрузке выделяются буферы вычислений, служебные структуры и иногда копии тензоров. Prefill с длинным входом может иметь отдельный пик временной памяти. При обслуживании нескольких запросов менеджер хранит свободные блоки и распределяет KV не так, как непрерывный массив в формуле.

PagedAttention³ разбивает логический кэш на блоки, уменьшая потери из-за выделения и позволяя делить общие префиксы. Значит, нижняя формула по токенам сохраняет смысл как полезные данные, но фактическая выделенная память зависит от размеров блоков и политики планировщика. Сначала измеряют холодный запуск, затем несколько реальных последовательностей с разными длинами и оставкой генерации.

Когда веса не входят в VRAM

Big Model Inference² показывает размещение весов по устройствам и выгрузку. Перенос части слоёв в RAM иногда позволяет запустить checkpoint при меньшей VRAM. Но рост времени ответа может сделать эту конфигурацию непригодной; надо измерить throughput и задержку на длительной генерации. Диск как резервное хранилище весов ещё сильнее отличается от быстрой памяти ускорителя.

Реальные форматы проверяют по документам runtime, например llama.cpp⁴. «Q4» не эквивалентно $P/2$ байт на диске: тип блоков и сохранённые в другой точности тензоры добавляют накладные данные. Результат квантования оценивают и по качеству ответа, особенно для языков и задач собственного корпуса.

Три измерения одной гипотезы

Выберите точный checkpoint и запишите фактический размер файла, объём VRAM после загрузки и пик при запросе. Повторите при 512 и 4096 входных токенах, затем при одном, четырёх и двадцати активных запросах. Сравните наблюдения с $W_0 + K$, перечислите источники расхождения, проверьте отсутствие скрытой выгрузки. Лишь после такого опыта можно сказать, что данная модель помещается и работает в выбранном режиме.

Для устройства с условными 16 GiB доступной VRAM сумма сырых весов и двадцати полных KV-последовательностей оставила бы около 2,27 GiB под остальные расходы. Этого остатка может не хватить; для устройства с 6 GiB тот же режим уже превышает ёмкость до учёта буферов. Поэтому вывод «8B в четырёх битах точно подходит» нельзя сделать из размера файла. Можно уменьшить конкурентность, контекст или расход кэша, но каждое изменение нужно проверить по качеству и задержке.

Решение о вместимости

Для двадцати полных кэшей сырая сумма 13,73 GiB. На условных 16 GiB остаётся 2,27 GiB. Если пик буферов выше, опыт завершится OOM или вытеснением.

Три наблюдения

Файл

Размер квантованных весов.

Покой

VRAM после загрузки.

Нагрузка

Пик, p95 и факт offload при целевых запросах.

Проверить источник

PagedAttention³, §2–3: почему реальная выделенная память отличается от простого массива. Разделить логические токены и физические блоки.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Hugging Face (документация, 2026), *Caching*. Документация.

Кэширование ключей и значений при генерации

Показаны варианты KV-кэша и ограничения отдельных реализаций: динамическое размещение, статический кэш и перенос. Документация помогает проверить, применяется ли простая формула полного кэша к выбранной архитектуре.

Что читать. Introduction; Cache Types; Offloaded Cache. Выписать поддерживаемый тип для конкретного checkpoint.

2. Hugging Face (документация, 2026), *Big Model Inference*. Документация.

Инференс моделей, не помещающихся целиком в ускоритель

Разъясняет dispatch весов между GPU, RAM и диском. Такая выгрузка может сделать запуск возможным, но пропускная способность пути передачи меняет практическую скорость.

Что читать. Big Model Inference и описание device map; различить факт загрузки и измеренную скорость.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Kwon, W. и др. (2023), *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP.

PagedAttention: управление памятью при обслуживании LLM

Работа вводит блочное размещение KV-кэша и показывает, как фрагментация и совместное использование влияют на пропускную способность обслуживания. Логическое число токенов и реально занятая физическая память различаются.

Что читать. §2 о проблеме памяти; §3 PagedAttention; §4 механизм vLLM и эксперименты.

4. ggml-org (документация, 2026), *Model files and quantization in llama.cpp*. Документация и код.

Файлы моделей и квантование в llama.cpp

Описаны форматы моделей и варианты запуска. Обозначение четырёхбитной квантизации относится к конкретной схеме блоков и метаданных, поэтому размер файла не обязан равняться идеальному четырём битам на параметр.

Что читать. Разделы о GGUF, quantization и loading; сравнить размер файлов одинакового checkpoint в двух форматах.

Порядок чтения. Начать с видов KV-кэша, затем изучить загрузку большой модели. PagedAttention объясняет распределение при конкуренции, документация llama.cpp — фактический формат весов. На выбранной модели сопоставить формулу с пиковой памятью.

3. Обучение и дообучение: от узла к кластеру

Обновление весов добавляет градиенты, состояния оптимизатора и сохранённые активации. Экономия памяти меняет обмен данными, время шага и результат обучения.

Почему память инференса нельзя переносить на обучение

При инференсе хранятся веса, KV и рабочие буферы. Полное обучение дополнительно требует градиенты, состояния оптимизатора, активации для обратного прохода и временные тензоры. Для учебной плотной модели с 8 миллиардами параметров только 16-битные веса заняли бы около 14,9 GiB; сумма с градиентами, мастер-копиями и состояниями оптимизатора может быть намного больше. Точное число определяется выбранным оптимизатором и форматом, поэтому правило «столько-то байт на параметр» без этих условий опасно.

Для заданного примера с 16-битными весами и градиентами и двумя 32-битными моментами Adam можно выделить учётную нижнюю сумму состояний:

$$M_{\text{states}} = P(2 + 2 + 4 + 4) = 12P \text{ байт.}$$

При $P = 8 \cdot 10^9$ это 96 GB, или около 89,4 GiB, ещё без активаций, мастер-копии весов, буферов и резервов. Число не следует переносить на другой оптимизатор или точность. Оно показывает, почему четырёхбитная оценка весов для инференса не является бюджетом обучения.

Увеличение длины последовательности и числа примеров в микро-батче повышает расход активаций. Gradient accumulation уменьшает размер одного шага на устройстве, сохраняя больший эффективный batch, но добавляет проходы и не устраняет память весов. Activation checkpointing пересчитывает часть операций на обратном проходе и меняет время на память. Прежде чем покупать устройство, определите, какие параметры действительно требуется обновлять.

Полный шаг, адаптер и квантованная база

Полное дообучение обновляет все веса и требует соответствующих градиентов и оптимизатора. LoRA замораживает базу и обучает низкоранговые добавки к выбранным слоям; активации и проход через базу всё равно нужны. QLoRA⁴ размещает замороженную базу в квантованном виде и обучает адаптеры. Это позволяет снизить расход под веса и обучаемые состояния, но не делает активации или пик временных буферов бесплатными.

Для задачи документов сравните три вопроса: можно ли достичь цели правильным извлечением без обучения; если нет, поможет ли небольшой адаптер; если нужен полный сдвиг модели, хватит ли данных и вычислительного бюджета. Улучшение обучающей потери не доказывает качество на новых документах. Создайте отложенный набор до запуска и зафиксируйте метрики цитирования и задач пользователя.

Состояния 8B по шагам

Веса FP16

16 GB.

Градиенты FP16

16 GB.

Два момента FP32

32 + 32 GB.

Итого

96 GB $\approx$ 89,4 GiB. Активации и прочие массивы добавляются.

Различающий вопрос

Ошибки ответа вызваны параметрами модели или отсутствием нужного фрагмента? Во втором случае сначала исправить поиск.

Читать

QLoRA⁴, §3: какие тензоры заморожены, какие обучаются и что остаётся в памяти.

Несколько GPU и стоимость синхронизации

Простейший data parallelism держит реплику модели на каждом GPU и объединяет градиенты. Если один экземпляр не входит в память одного устройства, дополнительные реплики сами по себе не помогают. FSDP² и ZeRO³ разделяют параметры, градиенты и состояния оптимизатора между устройствами; необходимые данные собираются по мере шага. Общая доступная память растёт, но ускорители начинают ждать обмена.

Даже если бы все 96 GB состояния идеального примера поровну распределились между двумя GPU, вышло бы 48 GB на каждый до активаций и временного восстановления тензоров. FSDP и ZeRO отличаются временем жизни полных параметров и набором шардируемых состояний. Реальная потребность определяется профилем шага; арифметическое деление общей памяти на два не даёт требования к каждой карте.

NCCL² описывает коллективные операции, например all-gather и reduce-scatter. Их стоимость зависит от топологии: связь внутри одного узла и между узлами имеет разную полосу и задержку. Tensor и pipeline parallelism добавляют другие границы обмена и распределения работы. Для кластера нужны скорость хранилища checkpoint, сеть, управление задачами, мониторинг и пробное восстановление, а не только число ускорителей.

Сквозная экономика эксперимента

План полного обучения включает оценку токенов и шагов, времени шага на прототипе, объёма checkpoint, частоты сохранения и вероятного повторного запуска. Если 8 GPU повышают throughput не в 8 раз, разница может уходить в обмен, дисбаланс или чтение данных. Масштабирование лучше измерять в *полезных* токенах за час при одинаковом качестве и плане обучения.

В локальной задаче сначала возьмите малую открытую модель и проведите один воспроизводимый шаг. Измерьте пик памяти и время для полного обновления, LoRA и QLoRA на одинаковой выборке. Затем отдельно оцените качество, поскольку самый компактный режим может иметь иные ограничения. Если обучение выполняется раз в полгода, удалённые вычисления на короткий период могут быть практичнее постоянно включённого кластера; такое решение требует учесть локальность данных.

Что сохранять после сбоя

Одних весов недостаточно для точного возобновления. Нужны состояния оптимизатора, планировщика, генераторов случайности, позиция в данных и сведения о версии кода. Проверьте восстановление в отдельном запуске, иначе высокая скорость обучения на бумаге может обернуться повторением всего эксперимента. Для адаптеров фиксируйте и точную ревизию базовой модели: без неё файл добавок неоднозначен.

Два GPU: нижняя оценка

96 GB состояний / 2 = 48 GB на устройство только при идеальном шардировании этих состояний. Активации, собранные на время слои и обмен — сверху.

Путь шага

Собирать

All-gather нужных параметров.

Считать

Прямой и обратный проход.

Разделить

Reduce-scatter градиентов и обновление.

Разобрать по источникам

FSDP²: пример одного шага;
NCCL²: семантика коллективных операций.

Проверить сбой

Возобновить шаг из checkpoint после прерывания на другом узле.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. PyTorch (документация, 2026), *Getting Started with Fully Sharded Data Parallel (FSDP2)*. Документация.

Полное шардирование параметров, градиентов и оптимизатора

Разделяет репликацию модели и шардирование состояний обучения. Описаны коллективные операции, временное восстановление весов и компромисс между памятью и коммуникацией.

Что читать. Overview; How FSDP2 works; all-gather и reduce-scatter. Схематично изобразить один шаг на двух GPU.

2. NVIDIA (документация, 2026), *NCCL Developer Guide*. Документация.

Коллективные операции NCCL при распределённом обучении

Показывает коллективные операции и коммуникационные топологии между GPU. Для multi-GPU конфигурации скорость межсоединения и доступный набор маршрутов важны наряду с суммой VRAM.

Что читать. Overview и Collective Operations; сравнить all-reduce, all-gather и reduce-scatter для градиентов.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Rajbhandari, S. и др. (2020), *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. SC20.

ZeRO: уменьшение памяти обучения разделением состояний

Авторы разбирают память оптимизатора, градиентов и параметров и этапы их разделения между процессами. Выигрыш памяти оплачивается обменом данными и зависит от топологии кластера.

Что читать. §2 о памяти состояний; §3 этапы ZeRO; проверить, что именно шардируется на каждом этапе.

4. Dettmers, T. и др. (2023), *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS.

QLoRA: эффективное дообучение квантованных LLM

Замороженная квантованная база сочетается с обучаемыми низкоранговыми адаптерами. Работа объясняет экономию памяти и цену активаций; число параметров адаптера само по себе не определяет весь пик VRAM.

Что читать. §3 QLoRA, quantization и paged optimizers; §4 эксперименты. Восстановить состав памяти для одного шага.

Порядок чтения. FSDP2 вводит распределённый шаг, NCCL — его операции обмена. ZeRO даёт модель памяти шардирования. QLoRA показывает другой путь снижения расхода: заморозить базу и обучать адаптер. Сопоставить все четыре подхода на одной задаче.

4. Инференс: от одного запроса к сервису

Скорость ответа складывается из поиска, обработки входа, генерации и ожидания в очереди. Пиковые токены в секунду не описывают поведение сервиса под нагрузкой.

Две фазы генерации

При prefill модель обрабатывает входную последовательность и создаёт начальное KV-состояние. При decode она последовательно генерирует токены, дополняя это состояние. Большой вход влияет прежде всего на время первого токена (TTFT), длинный ответ — на длительность decode. Поисковая стадия и очередь дополняют задержку, которую видит пользователь. Если в отчёте указан только средний throughput, трудно понять, почему интерактивный ответ медленный. Для учебного RAG-запроса измерьте отдельно поиск и reranking, токенизацию, prefill, первый токен и следующие 128 токенов. Удвоение числа приложенных фрагментов меняет вход и может улучшить либо ухудшить качество; это не бесплатная настройка. На том же GPU одновременные запросы могут улучшить суммарную загрузку, но увеличить ожидание и KV-память.

Планировщик и память

vLLM¹ служит примером сервера, где политика пакетирования и выделения KV связаны. PagedAttention³ уменьшает потери при переменных длинах запросов и поддерживает совместное использование префиксов. Однако блоки кэша конкурируют с весами и временными буферами. При слишком длинных входах или большом числе активных генераций сервер может вытеснять, пересчитывать либо отклонять работу в зависимости от настроек и версии.

Полезно отличать число зарегистрированных пользователей, число запросов в секунду и *одновременно активных последовательностей*. Двадцать сотрудников могут обращаться по очереди; двадцать параллельных длинных генераций создают другой пик памяти. Для определения этого пика соберите распределение поступления запросов, а не умножайте число учётных записей на максимальное контекстное окно.

Метрики с единицами

TTFT — время до первого токена; надо уточнить, включает ли измерение очередь и поиск. TPOT или ITL описывают темп выдачи последующих токенов. Для учебного ответа из n токенов приближённо разложим наблюдаемую задержку:

$$T_{\text{user}} \approx T_q + T_r + T_p + (n - 1)t_d + T_{\text{post}}.$$

Здесь T_q — очередь, T_r — поиск, T_p — prefill до первого токена, t_d — средний интервал дальнейшей генерации. Измеренная TPOT для разных запросов может различаться; это учебная декомпозиция, а не закон производительности. p95 показывает хвост, скрытый средним. Для сравнения фиксируют модель, длины и версии ПО.

Один ответ по шагам

Условно: очередь 0,4 s; поиск 0,2; prefill 0,8; ещё 128 интервалов по 0,04; постобработка 0,1.

Первый токен

После $0,4 + 0,2 + 0,8 = 1,4$ s для пользователя.

Полный ответ

$1,4 + 128 \cdot 0,04 + 0,1 = 6,62$ s.

Что измерять

Benchmark CLI²: TTFT и TPOT; отдельно добавить время поиска и очереди своего приложения.

Оговорка

Все числа условные. Средняя TPOT не определяет p95 ответа.

Как ставить нагрузочный опыт

Benchmark CLI vLLM² задаёт измеримый генератор запросов. Сначала прогрейте систему и отдельно измерьте одиночный запрос с 512 входными и 128 выходными токенами. Повторите для входа 4096 и ответа 512. Затем задайте частоту поступления и сравните p50/p95 TTFT, TPOT, число завершений, пик VRAM и ошибки. Синтетические токены полезны для контроля длины, но реальный корпус может давать другие префиксы, распределение и поиск.

Изменяйте один параметр за опыт: batch, число параллельных последовательностей, размер блоков KV либо режим offload. Иначе нельзя сказать, из-за чего изменилась скорость. Сохраните конфигурацию и журнал запуска: «на этой машине получилось 42 токена/с» без режима нагрузки и длины текста не является переносимым результатом.

Разделять prefill и decode или нет

DistServe⁴ исследует отдельные группы GPU для фаз prefill и decode. Если длинные входы мешают выдаче токенов другим запросам, разделение может улучшить соблюдение целевых задержек. При этом KV нужно передать между группами, и дорогое межсоединение входит в стоимость решения. Для локальной станции с одним GPU такой путь обычно не относится к ближайшему выбору; для многопользовательского сервера становится исследовательским вариантом.

Другие варианты — отдельные реплики под длинные и короткие запросы, ограничение длины, очередь с приоритетом, динамическое пакетирование. Их сравнивают при одинаковом качестве и полной цене. Более короткий контекст может уменьшить расход, но терять нужный фрагмент. Более сильное квантование может улучшить вместимость, но изменить ответы. Аппаратное решение невозможно отделить от политики сервиса.

Когда инцидент похож на нехватку железа

OOM при загрузке, OOM только на длинных запросах и рост p95 без OOM — разные случаи. В первом проверяют веса и формат; во втором KV и пик prefill; в третьем очередь, CPU, сеть, поиск и частоты устройств. Отчёт об обслуживании должен разделять эти симптомы. Покупка GPU оправдана после того, как протокол показал узкое место, которое дополнительные ресурсы способны устранить.

Зафиксируйте полезную производительность как число запросов, удовлетворивших заранее заданным ограничениям времени и качества за период измерения. Сервер может завершить больше запросов, но хуже выполнить предел TTFT; в таком случае один throughput не отражает успех. Не меняйте критерий после получения замеров. Для длинных ответов проверьте отдельно TPOT и полное время: хорошее начало не гарантирует удобного окончания.

Полезная скорость

Условно за 10 с завершено 100 запросов, 80 отвечают порогам TTFT, TPOT и качества.

Goodput в этом протоколе — 8 подходящих запросов/s, не 10.

Различающий тест

Вход 512/4096; ответ 128/512; конкурентность 1/4/20.

Фиксировать checkpoint и распределение поступления.

Когда читать DistServe

DistServe⁴, §2–4: измерить помехи между prefill и decode и цену пересылки KV, прежде чем разделять GPU.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. vLLM (документация, 2026), *vLLM Documentation*. Документация и код.

Сервер инференса vLLM и его планировщик

Объединяет планирование запросов, управление KV-памятью и запуск сервера. Конкретные параметры зависят от версии; исходный runtime необходимо фиксировать вместе с измерениями.

Что читать. Serving и Configuration; проследить вход запроса до выделения KV-блоков и ответа.

2. vLLM (документация, 2026), *Benchmark CLI*. Документация.

Измерение времени первого токена и времени между токенами

Документирует параметры генератора нагрузки и метрики TTFT, TPOT, throughput и задержки. Сопоставление машин требует одинаковой модели, распределения длин и интенсивности поступления запросов.

Что читать. Benchmark Serving и определения метрик; составить команду опыта с воспроизводимыми входными длинами.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Kwon, W. и др. (2023), *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP.

PagedAttention: управление памятью при обслуживании LLM

Работа вводит блочное размещение KV-кэша и показывает, как фрагментация и совместное использование влияют на пропускную способность обслуживания. Логическое число токенов и реально занятая физическая память различаются.

Что читать. §2 о проблеме памяти; §3 PagedAttention; §4 механизм vLLM и эксперименты.

ФРОНТИР ИССЛЕДОВАНИЙ

4. Zhong, Y. и др. (2024), *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving*. OSDI.

DistServe: раздельное исполнение prefill и decode

Предлагается разместить фазы prefill и decode на разных группах GPU, чтобы уменьшить помехи и отдельно выбирать их параллелизм. Передача KV-состояния тоже имеет цену, поэтому результат зависит от нагрузки и сети.

Что читать. §2 анализ помех; §3 архитектура; §4 оптимизация размещения и измерения goodput.

Порядок чтения. Документация vLLM описывает сервер, PagedAttention — управление кэшем. Затем изучить определения метрик в Benchmark CLI и воспроизвести опыт. DistServe читать после понимания фаз, оценивая стоимость передачи состояния.

5. Данные, индексы и базы для RAG

Надёжная документная система тратит память и диск на исходники, тексты, векторы, индексы, метаданные и версии. Качество поиска измеряется отдельно от генерации.

От файла до проверяемого фрагмента

Корпус PDF и таблиц поступает на диск. Извлечение и OCR создают текст с привязкой к странице и документу. Чанкирование даёт фрагменты, эмбединги — векторы; база хранит их идентификаторы, разрешения, версию и путь к исходнику. Если документ обновлён, старый вектор может продолжать отвечать на вопросы, пока операция переиндексации не выполнена согласованно.

Introduction to Information Retrieval¹ напоминает, что релевантность надо проверять на наборе вопросов. Для RAG пригодна параллельная оценка: попал ли нужный фрагмент в top-k, можно ли процитировать правильную страницу, и сохраняется ли ответ при изменении документа. Хороший ответ генератора не доказывает хорошего индекса: модель могла угадать, опираясь на уже известный ей факт.

Оценка сырых векторов

При N векторов размерности d , хранимых в s байтах на компонент, нижняя граница равна $V_0 = Nds$. Для 150,000 фрагментов по 1024 компонента FP32 получим 614,400,000 байт, приблизительно 0,572 GiB. Это только числа векторов. Текст, метаданные, индексные связи, структуры БД, WAL и резервная копия добавляются отдельно. Если размерность или формат меняется, весь расчёт надо повторить. pgvector² позволяет сравнивать точный поиск с HNSW и IVFFlat внутри PostgreSQL. Точный поиск служит эталоном recall на маленькой проверочной выборке. Приближённый индекс ускоряет запросы ценой дополнительной памяти и вероятности пропуска соседа. SQL-фильтр по доступу и версии может резко уменьшить выборку и изменить эффективный план запроса; векторный бенчмарк без такого фильтра не описывает реальную задачу.

Где должен стоять SSD

Размер файла векторов не равен диску под БД. Учтите исходники, извлечённый текст, индексы, временное место перестроения, транзакционные журналы и копии. Для горячего индекса важны RAM и характер доступа; для редкого архива — ёмкость и скорость восстановления. Не размещайте единственную резервную копию на том же отказавшем устройстве.

Корпус по шагам

1. Вектор

$150000 \times 1024 \times 4$ байта = 0,572 GiB для FP32.

2. Сжатие

FP16 дал бы 0,286 GiB сырых чисел; точность поиска проверять отдельно.

3. Остальное

Текст, граф HNSW, метаданные, WAL и резерв не включены.

Вопрос к индексу

Есть ли нужная страница в top-k после фильтра по доступу и версии? pgvector², Filtering и Exact Search.

Граф индекса и граф знаний

HNSW³ строит граф близости между векторами для приближённого поиска. Рёбра отражают геометрию эмбедингов и путь поиска кандидатов. Это не граф предметных фактов: отношение «сотрудник отвечает за проект» надо хранить и проверять отдельно. Рост параметров построения и поиска меняет компромисс между recall, временем и памятью; универсального значения для каждого корпуса нет. Сравнение поиска ставят на фиксированном списке вопросов с размеченными нужными фрагментами. Измеряют recall@k относительно эталона, р95 времени с учётом фильтров и пик RAM при построении и обновлении. Результат генерации проверяют отдельно: правильный фрагмент может быть извлечён, но проигнорирован моделью; плохой фрагмент может получить уверенную ссылку. Реранжирование добавляет ещё одну модель и свой бюджет CPU/GPU.

Обновление и восстановление

Документация PostgreSQL по PITR⁴ объясняет согласованную резервную копию и WAL. Но восстановление одной таблицы векторов не возвращает автоматически те PDF и модели эмбедингов, из которых она была построена. Для нашей системы запишите version ID исходника, конфигурацию OCR, правила чанкирования, идентификатор модели и версию индекса. После изменения любого из них переиндексация должна иметь явный статус.

Учебный тест отказа: снимите резервную копию, добавьте и измените два документа, удалите один, затем восстановитесь на отдельной машине. Проверьте не только число строк, но и доступность исходных страниц, разрешения и ответы по трём контрольным вопросам. Восстановление, которое ни разу не испытывали, остаётся предположением.

Системный бюджет

Рассчитывайте отдельно (1) исходники и архив, (2) рабочий корпус с индексом и журналом, (3) модели поиска и генерации с их оперативной памятью. Это помогает понять, когда добавление SSD решает проблему, а когда нужен объём RAM или GPU. Для крупного многопользовательского корпуса учитывайте сетевую передачу, резервирование БД и время построения нового индекса без остановки сервиса. Проведите парное сравнение: на одном наборе вопросов запишите точную выдачу, затем приближённую при равных фильтрах, k и версии корпуса. Разность во времени без измерения пропущенных нужных фрагментов мало говорит о пользе индекса. После изменения OCR или embedding повторите оценку: прежние векторы могут больше не соответствовать тому же тексту. Результаты индексации сохраняются вместе с конфигурацией преобразований.

Обновление одного PDF

До

Страница 7, фрагмент А, embedding версии 1.

После

Страница 7 изменилась: новая версия текста, фрагмента и вектора; старый ответ не должен сослаться на А.

Проверка

Запрос со старой цитатой после переключения индекса.

Где разобрано

HNSW³, §3: связи графа соседей. PITR⁴: база и WAL. Версию исходного PDF сохранять отдельно.

Различие

Граф близости векторов, граф фактов и граф выполнения задачи отвечают на разные вопросы.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Manning, C. D.; Raghavan, P.; Schütze, H. (2008), *Introduction to Information Retrieval*. Учебник.
Введение в информационный поиск
Объясняет инвертированные индексы, ранжирование и оценку поиска. Качество RAG начинается с выбора релевантных фрагментов, а не с объёма VRAM генератора.
Что читать. Главы 1, 6 и 8: индекс, ранжирование и оценка; задать тестовый набор запросов.
2. pgvector (документация, 2026), *pgvector: Open-source vector similarity search for Postgres*. Документация и код.
Векторный поиск в PostgreSQL
Описаны точный поиск, HNSW и IVFFlat, типы векторов, индексы и SQL-фильтрация. Выбор индекса отражается на RAM, размере диска, времени построения и качестве поиска.
Что читать. Exact Search; HNSW; IVFFlat; Filtering. Замерить recall относительно точной выдачи после фильтра.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

3. Malkov, Y. A.; Yashunin, D. A. (2016), *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*. Исследовательская статья.
HNSW: приближённый поиск по иерархическому графу близости
Иерархический граф позволяет быстро находить кандидатов, меняя время и память на достигаемую полноту. Векторный индекс хранит связи между точками помимо самих чисел вектора.
Что читать. §3 построение и поиск; §4 эксперименты. Сравнить стоимость индекса и recall с точным поиском.
4. PostgreSQL Global Development Group (документация, 2026), *Continuous Archiving and Point-in-Time Recovery (PITR)*. Документация.
Непрерывное архивирование и восстановление базы на момент времени
Резервная копия и журнал WAL позволяют восстановить согласованное состояние. Для RAG надо также согласовать версию исходного документа, фрагментов, эмбедингов и индекса.
Что читать. Setting Up WAL Archiving; Making a Base Backup; Recovering. Провести пробное восстановление на отдельном узле.

Порядок чтения. Учебник по IR задаёт качество поиска. pgvector показывает исполнимые альтернативы, HNSW — устройство приближённого индекса. PITR завершает путь: восстановить не только записи, но и согласованные производные данные.

6. Локальные модели: от SLM до LLM

Локальный запуск определяется совместимостью checkpoint, формата и runtime с устройством. Размер модели, квантизация и разгрузка слоёв выбираются вместе с качеством и задержкой.

Выбрать конкретную модель

Начинают не с абстрактного «7B», а с точного идентификатора модели и ревизии. Проверяют задачу, языки, размер контекста, лицензию, известные ограничения, формат файлов и tokenizer. Карточка модели¹ — отправная точка, но заявленная способность не заменяет собственный тест на документах. Небольшая SLM может быть достаточной для классификации или извлечения, тогда как длинный синтез по нескольким источникам предъявляет другие требования.

Для нашего проекта полезно разделить роли: OCR, embedding, reranker и генератор не обязаны быть одним и тем же checkpoint. Их можно запускать последовательно на одном устройстве, если требуемая задержка позволяет выгрузку, или держать постоянно, если потоки запросов пересекаются. Сумма файлов на диске и одновременный пик памяти — разные величины.

Три вопроса совместимости

Первый: поддерживает ли runtime архитектуру, tokenizer, chat template и нужные модальности. Второй: поддерживает ли backend данного устройства операции, формат весов и точность. Третий: помещаются ли веса, KV и рабочие буферы при целевом контексте. llama.cpp² показывает практический локальный маршрут CPU/GPU с GGUF и сервером; его поддержка конкретных новых моделей проверяется по коду и пробному запуску, а не по общему названию проекта.

На условном устройстве с 6 GiB VRAM сырые веса нашего 8B при четырёх битах (3,73 GiB) и полный KV одной последовательности (0,5 GiB) оставили бы не более 1,77 GiB под метаданные и рабочие буферы. Это кандидат на испытание, а не обещание запуска. Для двадцати последовательностей сырой бюджет уже выше 6 GiB. Для условной плотной 32B одни четырёхбитные веса потребуют около 14,9 GiB, и полный перенос на такую GPU невозможен. CPU/offload может помочь запуску ценой задержки. Небольшую SLM, выбранную по задаче, также нужно испытать, а не считать годной по числу параметров.

Данные локальны только при полном контуре

Локальный генератор не делает систему локальной, если OCR, embedding или телеметрия отправляют документы во внешний сервис. Проверьте каждый процесс, обновления, загрузку весов, лицензии и хранение журналов. В паспорте машины должны быть и политика резервных копий, и возможность восстановить окружение без сети, если этого требует задача.

6 GiB: условный опыт

Один запрос

3,73 GiB весов + 0,5 GiB KV = 4,23 GiB. Остаток до 6 GiB — 1,77 GiB до буферов.

Двадцать

Сырой бюджет 13,73 GiB; этот режим не поместится целиком в 6 GiB.

32B Q4

Около 14,9 GiB одних идеальных весов.

Проверить по карточке

Model Cards¹: ID, ревизия, лицензия, tokenizer и допустимые задачи; затем llama.cpp²: поддержка формата.

Разные программные экосистемы

У GPU NVIDIA обычно проверяют CUDA, compute capability, драйвер и соответствующую сборку приложения. Для AMD матрица ROCm³ сопоставляет устройство и версии ОС, фреймворка и runtime; поддержка обозначения серии не равна проверке точной карты и модели. На Apple Silicon MLX⁴ использует общую память CPU/GPU. Это снимает некоторые расходы копирования, но не превращает системную RAM в неограниченную VRAM: ёмкость и bandwidth остаются конечными, а другие процессы тоже расходуют ресурс.

CPU-only полезен как baseline и иногда достаточен для малой модели и низкой частоты запросов. При выборе CPU смотрят на производительность одного потока, векторные инструкции, bandwidth RAM и время полной генерации. Для ноутбука важны также ограничения мощности и охлаждения: краткий бенчмарк не описывает час непрерывной работы. Для дискретного GPU проверьте физическую совместимость, питание и сколько слоёв runtime действительно положил на ускоритель.

От загрузки к воспроизводимому запуску

Сохраните hash или ревизию файлов, команду запуска, лимит контекста, размер batch, число GPU-слоёв либо схему размещения, версии драйвера и runtime. Прогрейте модель. Проведите три набора вопросов: короткие факты, длинные документы и случаи, когда нужно признать отсутствие ответа. Сравните точность цитат и выходов в исходной и квантованной версии, затем скорость и память.

Если квантованная модель влезает, но результат ухудшается на обязательных задачах, выбор не пройден. Если точная модель работает лишь с выгрузкой в RAM и длительное ожидание недопустимо, она тоже не подходит. Иногда модель меньшего размера с лучшим retrieval побеждает более крупную локальную модель при том же бюджете; это проверяется на корпусе, а не предполагается по размерам.

Путь к собственному дообучению

Для локального LoRA сначала зафиксируйте базовый checkpoint, обучающие и отложенные данные, лицензию и формат адаптера. Оцените память по предыдущей главе, сделайте один пробный шаг и измерьте качество после адаптации. Убедитесь, что runtime умеет применить адаптер или что процедура слияния даёт проверенный артефакт. Обучение и запуск часто используют разные стеки — совместимость результата нужно проверить отдельно.

Результат маршрута — таблица из четырёх строк: модель и ревизия; формат и backend; пик памяти на целевой длине; качество и p95. Если один столбец неизвестен, итог «совместимо» преждевременен. Повторите запуск после обновления стека: изменение драйвера, квантователя или chat template может изменить как скорость, так и содержание ответа.

Разные backend

NVIDIA

Проверить compute capability, драйвер и нужные ядра выбранного runtime.

AMD

Матрица ROCm³: точный GPU, ОС и версия фреймворка.

Apple

MLX⁴: общая память, поддержка модели и длительная скорость.

Тест результата

Одна ревизия и набор вопросов: исходная и квантованная версия, цитаты, пик памяти и p95. Затем проверить адаптер после отдельного обучения.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. Hugging Face (документация, 2026), *Model Cards*. Документация.

Карточки моделей: лицензия, назначение и ограничения

Карточка связывает веса с задачей, происхождением, условиями использования и известными ограничениями. Это начало проверки, а не измерение пригодности на собственных документах. Для воспроизводимого запуска отдельно зафиксируйте ревизию checkpoint, tokenizer, шаблон диалога и лицензию.

Что читать. Overview и metadata; прочитать карточку выбранной ревизии, затем сверить конфигурацию tokenizer и провести задачу на отложенных примерах.

2. ggml-org (документация, 2026), *llama.cpp*. Документация и код.

llama.cpp: локальный запуск и сервер моделей

Проект показывает локальный запуск CPU/GPU, частичную выгрузку слоёв, сервер и параметры контекста. Поддержка backend не гарантирует, что точный checkpoint корректно загрузится или достигнет нужной скорости. Запишите команду, ревизию весов и число фактически выгруженных на GPU слоёв.

Что читать. README, build instructions и server README; запустить выбранный checkpoint на CPU и GPU, сравнить размещение, первый токен и устойчивую генерацию.

3. AMD (документация, 2026), *ROCm Compatibility Matrix*. Документация.

Матрица совместимости ROCm

Матрица сопоставляет конкретные устройства, ОС, версии ROCm и поддерживаемые фреймворки. Проверка только серии GPU оставляет неопределёнными сборку runtime и требуемые операции модели. Сохраните выбранную строку матрицы рядом с протоколом фактического запуска.

Что читать. GPU, OS and framework support именно своей версии; отметить подтверждённые и неизвестные сочетания до установки стека.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

4. Apple ML Explore (документация, 2026), *MLX*. Документация и код.

MLX: вычисления на Apple Silicon

Описывает стек для CPU и GPU Apple Silicon с общей памятью и собственным API. Общая память облегчает обмен данными, но приложения и система делят конечную ёмкость; bandwidth и тепловой режим также ограничивают длительную выдачу. Указание поддержки MLX само по себе не проверяет конкретную модель.

Что читать. README и документация о device/array; проверить формат выбранных весов, пик общей памяти и скорость генерации после прогрева.

Порядок чтения. Прочитать карточку выбранной модели, затем документацию llama.cpp для воспроизводимого CPU/GPU запуска. ROCm и MLX читать по соответствующему устройству. Последним шагом провести собственный тест на одних и тех же вопросах.

7. Выбор компонентов и контрольный практикум

Итог — спецификация, в которой каждому компоненту соответствует измеренное требование. Решение принимают по качеству, p95 задержке, восстановлению и полной цене.

Заполнить паспорт, прежде чем выбирать детали

Для учебного проекта запишите: ревизии всех моделей; пять тысяч PDF и таблиц или реальный размер собственного корпуса; число фрагментов; распределение входа и выхода; целевую конкурентность; требования к ссылкам и защите; окно обслуживания. Оцените W_0 , KV и сырые векторы, добавив явно помеченный запас для runtime, индекса и роста. Затем измерьте текущий baseline хотя бы на малой модели: без него трудно судить, что даст новая покупка.

Разбейте спецификацию по компонентам. GPU: поддержка численного формата и ядер, VRAM, bandwidth, устойчивость под нагрузкой. CPU: парсинг, очередь, токенизация, поиск и способность кормить GPU. RAM: корпус, индекс, offload и рабочие процессы. SSD: исходники, модели, WAL, временное переиндексирование и копии. Материнская плата: PCIe, слоты RAM, накопители и физическое размещение. БП и охлаждение: стабильная длительная работа. Сеть: удалённые пользователи, бэкап или межузловой обмен. Не складывайте ёмкость разных уровней памяти как если бы это был один быстрый пул.

Наш учебный режим из двадцати последовательностей требует 13,73 GiB только под идеальные веса и полный KV. Поэтому условная карта с 16 GiB оставляет 2,27 GiB до рабочего пика, а с 24 GiB — 10,27 GiB; ни одно число ещё не доказательство пригодности. Система с одним коротким запросом представляет другой класс нагрузки. Для RAM и SSD сначала измеряют реальный корпус и индекс: формула векторов не сообщает объём страниц PDF, текстов OCR и резервных копий.

Аппаратная совместимость — проверяемое дерево

Документация о compute capability¹ помогает установить поддерживаемые возможности NVIDIA GPU. Для AMD и Apple проверка строится по их собственным матрицам из предыдущей темы. Далее подтверждают ОС, драйвер, сборку runtime, точный checkpoint и операцию. Наконец, проверяют физическую посадку карты, разъёмы и тепловой режим. В таблице решения каждая строка имеет статус «проверено запуском», «документировано» или «неизвестно»; неизвестное не превращают в совместимое без испытания.

Порядок покупки

Составьте три класса: имеющаяся машина с малой моделью; одна локальная станция с достаточной памятью и надёжным хранилищем; сервер или временная аренда для большой конкурентности и обучения. Для каждого оцените полную цену владения на период работы. Покупка самого мощного компонента не решает задачу, если ограничение находится в индексе или CPU. Отложите точный список продаваемых SKU до проверки цен и актуальных драйверов на дату закупки.

Две условные ёмкости

При сырых весах + KV = 13,73 GiB:

16 GiB

Остаётся 2,27 GiB до буферов: высокий риск OOM.

24 GiB

Остаётся 10,27 GiB: нужен замер реального пика.

Чем подтвердить выбор

GPU

Compute capability¹ + запуск точного runtime.

CPU/RAM

Профиль парсинга и поиска; объём рабочего индекса.

SSD/сеть

Перестроение индекса, бэкап и целевая конкурентность.

Статус

Запуском проверено; документировано; неизвестно. Только первый вариант подтверждает полную связку.

Четыре опыта

Первый — память. Предскажите нижний расход для небольшой и более крупной модели при 1, 4 и 20 активных последовательностях. Снимите фактический пик, отличите память allocator от полезных тензоров с помощью PyTorch CUDA Memory³ там, где этот инструмент применим. Назовите, почему наблюдение отличается от формулы.

Второй — скорость. На одной модели и одинаковых входах сравните CPU, GPU и частичный offload. Используйте PyTorch Profiler² или соответствующий профайлер runtime, разделите чтение, поиск, prefill и decode. Замерьте короткие и длинные запросы, TTFT, TPOT, p95, мощность и ошибки. Один запуск без прогрева и указания версии не даёт надёжной сравнительной оценки.

Третий — RAG. На реальном срезе корпуса измерьте время OCR и embedding, размер текста и индекса, точный и приближённый поиск с одинаковыми фильтрами. Проверьте страницы цитат и восстановление после изменения документа. Четвёртый — обучение. На небольшой модели сделайте одинаковый цикл для полного обновления, LoRA и QLoRA; сохраните пик памяти, шаги/с, checkpoint и качество на отложенных примерах.

Чтение бенчмарков и решение

MLPerf Inference⁴ задаёт сценарии и правила сопоставления систем. Пользуйтесь внешними результатами как ориентирами, если совпадают задача, точность и ограничения. Для собственной закупки первичен полный эксперимент: качество не ниже требуемого, p95 в целевом режиме, устойчивость под длительной нагрузкой и стоимость за полезную задачу.

Для сравнения двух конфигураций запустите одни и те же вопросы на одном checkpoint и посчитайте парную разность задержек. Если $T_i(A)$ и $T_i(B)$ — полные времена ответа для вопроса i , то учебная оценка равна

$$\overline{\Delta T} = \frac{1}{n} \sum_{i=1}^n [T_i(B) - T_i(A)].$$

На четырёх условных вопросах справа получается минус три секунды: вариант B быстрее в среднем именно в этой маленькой выборке. Для вывода о p95, длительной устойчивости и покупке нужны гораздо больше реалистичных запросов, одинаковые настройки качества и повторные прогоны. Изменение модели или квантизации вместе с железом превратит опыт в сравнение пакетов решений, а не устройств.

Последний артефакт — таблица кандидатов со ссылкой на протокол запуска и причиной выбора. Если ни одна машина не укладывается, изменяйте измеримое требование: модель, retrieval, квантизацию, число одновременных запросов либо схему размещения. Не объявляйте устройство «подходящим» только потому, что файл модели загружается. Пробелы следующего этапа Атласа — лабораторные испытания конкретных конфигураций и рыночные цены на дату покупки, которые этот выпуск не подменяет.

Парное сравнение

Все числа условные, секунды полного ответа:

i	$T_i(A)$	$T_i(B)$
1	12	8
2	10	9
3	14	10
4	12	9

Разности: $-4, -1, -4, -3$; средняя $\overline{\Delta T} = -3$ s. Четыре вопроса не оценивают надёжно p95.

Что сохранить

Паспорт, расчёт, версии стека, журнал нагрузки, качество, измерения и причину выбора.

Граница

Опубликованный результат MLPerf⁴ относится к своему сценарию; цены и SKU проверять на дату покупки.

БИБЛИОГРАФИЯ

ВВОДНЫЕ РАБОТЫ

1. NVIDIA (документация, 2026), *CUDA GPU Compute Capability*. Документация.

Вычислительные возможности GPU NVIDIA

Возможности чипа связаны с набором операций и поддержкой численных форматов. Два ускорителя с одинаковой VRAM могут по-разному исполнять нужные ядра. Соответствие чипа таблице ещё не проверяет версии драйвера, сборку приложения и конкретный checkpoint.

Что читать. Таблицы compute capability и feature support; проверить точную архитектуру GPU, затем подтвердить её пробным запуском нужного runtime.

2. PyTorch (документация, 2026), *PyTorch Profiler*. Документация.

Профилирование вычислений и памяти в PyTorch

Профайлер показывает время операций, активность CPU/GPU и часть выделений памяти. Он позволяет проверить гипотезу об узком месте, например сравнить токенизацию с исполнением ядер, но собственные метрики сервера и поиск надо измерять отдельно.

Что читать. Profiler API и trace; измерить одинаковые запросы после прогрева, сопоставить CPU с GPU, затем добавить серверные TTFT и p95.

3. PyTorch (документация, 2026), *Understanding CUDA Memory Usage*. Документация.

Понимание расхода памяти CUDA

Snapshot и статистика allocator позволяют отделить память активных тензоров, резерв и пик во времени. Это помогает понять, почему нижняя оценка весов и KV не равна занятой VRAM. Аллокации вне видимости PyTorch надо учитывать иными средствами.

Что читать. Recording memory history и Visualizing snapshots; отметить, какие расходы инструмент не видит, и сравнить пик с расчётом главы.

СПЕЦИАЛИЗИРОВАННЫЕ РАБОТЫ

4. MLCommons (документация, 2026), *MLPerf Inference*. Бенчмарк и правила.

MLPerf Inference: сценарии и сопоставимые измерения

Набор задаёт сценарии, пороги точности и правила сравнения систем. Публичная цифра throughput полезна как внешний ориентир только после сверки модели, входных данных, ограничений задержки и настроек. Для закупки по документному RAG нужен собственный поток запросов и проверка качества ссылок.

Что читать. Scenarios, rules and results; выписать несовпадения со своей нагрузкой и разработать парный опыт на точном checkpoint.

Порядок чтения. Сначала установить возможности GPU, затем инструментами профилирования и памяти проверить свою систему. MLPerf читать как образец дисциплины сравнения, учитывая несовпадение задачи. Результат — воспроизводимое решение по классу машины.